

Digital Dub Siren

Raspberry Pi Zero 2 W Comprehensive Build Plan

A complete implementation plan for a seven-control digital dub siren: hardware selection, GPIO and ADC wiring, I2S audio, DSP design, operating-system setup, staged construction, verification and troubleshooting.

Version 1.0
18 July 2026

Contents

1. Project objective	3
2. Recommended result	3
3. System architecture	3
4. Why these parts	3
5. Bill of materials	3
6. Audio-output decision	4
7. Electrical rules	5
8. MCP3008 wiring	5
9. PCM5102A wiring	5
10. Buttons and indicator wiring	6
11. Control mapping	6
12. Sound engine	6
13. Software structure	7
14. Operating-system setup	8
15. Build phases	8
16. Test plan	9
17. Troubleshooting	9
18. Risks and mitigations	10
19. Definition of done	10
20. Future extensions	10
21. Recommended next action	10

1. Project objective

Build a compact, playable digital dub siren around a Raspberry Pi Zero 2 W. Seven physical knobs and several switches control a software synthesizer in real time. The Pi generates the oscillator, siren sweep, envelope, filter, echo and reverb, then sends stereo audio through a 3.5 mm jack to a mixer, active speaker or amplifier.

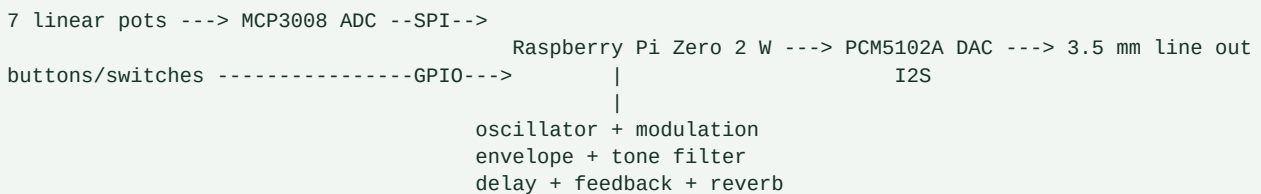
This is not a component-for-component recreation of the analogue 555 circuit. It recreates its musical function while making the sound engine easier to alter, save and extend.

2. Recommended result

Use a Raspberry Pi Zero 2 WH, an MCP3008 analogue-to-digital converter, and a PCM5102A I2S DAC module with line output. Prototype on a breadboard. Once the controls and software are stable, transfer the control wiring to perfboard or a custom PCB and install everything in an enclosure.

For the fastest first sound, use a small USB audio adapter instead of the PCM5102A. Replace it with the I2S DAC for the final compact build.

3. System architecture



Signal flow:

```

waveform oscillator -> pitch modulation -> envelope -> tone filter
                    -> dry/wet delay and reverb -> master gain -> DAC
  
```

4. Why these parts

- Raspberry Pi Zero 2 W: enough CPU for a small real-time stereo effects engine, Wi-Fi for development, and a compact 65 x 30 mm form factor. The standard Zero 2 W has an unpopulated GPIO header; the WH/header version avoids soldering the 40-pin connector.
- MCP3008: the Pi has no analogue inputs. This 10-bit, 8-channel SPI ADC reads seven knobs with one spare channel. [Microchip MCP3008 product page](#)
- PCM5102A: the Pi Zero has no analogue headphone jack. This DAC accepts I2S audio and provides good-quality stereo line output. [Texas Instruments PCM5102A product page](#)
- 10 kohm linear potentiometers: used as 3.3 V voltage dividers, so their exact resistance is not musically critical. Linear taper gives predictable control movement.
- Raspberry Pi OS Lite: minimal background load and easy headless administration over SSH.

The Raspberry Pi documentation confirms that all Pi models support USB audio. This makes a USB sound adapter a useful low-risk prototype option. [Raspberry Pi audio documentation](#)

5. Bill of materials

Prices are planning allowances for the Netherlands, including VAT where practical. Stock and shipping change. Enclosure, tools and shipping are excluded.

Qty	Item	Specification	Allowance	Example source
1	Raspberry Pi Zero 2 WH	512 MB, Wi-Fi, pre-soldered 40-pin header	EUR 21-30	Kiwi Electronics
1	microSD card	16-32 GB, reputable A1-rated card	EUR 7-12	Local electronics retailer
1	5 V power supply	Regulated, 2-2.5 A, micro-USB	EUR 8-15	Raspberry Pi-compatible supplier

Qty	Item	Specification	Allowance	Example source
1	MCP3008	DIP-16, 10-bit, 8-channel SPI ADC	EUR 4-7	DigiKey search
1	PCM5102A module	I2S input; stereo line output, preferably fitted 3.5 mm jack	EUR 6-15	DigiKey PCM5102A information
1	USB audio adapter	Optional prototype alternative with 3.5 mm output	EUR 7-15	Any Linux-compatible USB Audio Class adapter
1	Micro-USB OTG adapter	Required if using USB sound adapter	EUR 2-5	Raspberry Pi supplier
1	MCP3008 DIP socket	DIP-16	EUR 0.50-1	TME or DigiKey
7	Potentiometer	B10K/10 kohm linear, panel mount	EUR 7-18 total	TME, TinyTronics or Thonk
7	Knob	Matches selected shaft	EUR 7-20 total	TME, TinyTronics or Thonk
1	Momentary button	Normally open, panel mount	EUR 1-4	TME or TinyTronics
2	Toggle switches	SPDT ON-ON, panel mount	EUR 4-10 total	TME or DigiKey
1	LED	5 mm plus 330 ohm resistor	EUR 0.50	Any component supplier
1	Breadboard	Full-size, good contact quality	EUR 7-12	TinyTronics or Kiwi Electronics
1 set	Jumper wires	Female-to-male plus solid breadboard wire	EUR 6-12	TinyTronics or Kiwi Electronics
1	3.5 mm audio cable	Stereo TRS	EUR 3-8	Local audio retailer
1	Active speaker or mixer	Accepts line-level input	Existing equipment	Not included in total

Expected prototype electronics total: EUR 75-125, excluding tools, enclosure, active speaker and shipping.

6. Audio-output decision

Option A - USB audio adapter

Use this for the first prototype.

Advantages:

- No DAC wiring or I2S configuration.
- Usually includes a 3.5 mm headphone/line socket.
- Linux detects standard USB Audio Class devices automatically.

Disadvantages:

- Occupies the Pi Zero's only USB data port.
- Requires an OTG adapter or hub.
- Latency and noise vary by adapter.

Option B - PCM5102A I2S DAC

Use this for the final instrument.

Advantages:

- Low, consistent latency.
- Better integration and audio quality.
- Leaves USB available.
- Provides proper stereo line-level output.

Disadvantages:

- Requires GPIO wiring and software configuration.
- Module pin labels and hardware-mode jumpers vary.

- A bare PCM5102A line output cannot safely drive a passive speaker and may not drive demanding headphones.

The line output should connect to a mixer, powered monitor or external amplifier. If direct headphone use is required, select a USB adapter or codec board with an explicit headphone amplifier.

7. Electrical rules

1. Power the Pi from regulated 5 V only. Do not connect the original siren's 9-12 V supply to the Pi.
2. All GPIO and MCP3008 control signals are 3.3 V. Never place 5 V on a Pi GPIO pin.
3. Connect every potentiometer between 3.3 V and ground. Connect only its wiper to an ADC channel.
4. Tie analogue and digital grounds together at the MCP3008 and keep audio wiring away from the Pi power cable.
5. Add 100 nF decoupling close to the MCP3008 VDD pin. Add 10 uF across the breadboard's 3.3 V and ground rails.
6. Set software output gain conservatively. Delay feedback must be limited below self-oscillation unless deliberate.
7. Do not short the DAC's left and right channels together. For mono output, mix them through separate resistors.

8. MCP3008 wiring

Use BCM GPIO numbering in software.

MCP3008 pin	Name	Connection
1-7	CH0-CH6	Wipers of the seven pots
8	CH7	Spare expansion input
9	DGND	Pi ground
10	CS/SHDN	GPIO8, physical pin 24
11	DIN	GPIO10/MOSI, physical pin 19
12	DOUT	GPIO9/MISO, physical pin 21
13	CLK	GPIO11/SCLK, physical pin 23
14	AGND	Pi ground
15	VREF	Pi 3.3 V
16	VDD	Pi 3.3 V

Place a 100 nF ceramic capacitor between pins 16 and 14 close to the IC.

9. PCM5102A wiring

Typical module connections:

PCM5102A module signal	Pi connection
VIN	5 V or 3.3 V exactly as required by the selected module
GND	Pi ground
BCK/BCLK	GPIO18, physical pin 12
LCK/LRCK/WS	GPIO19, physical pin 35
DIN/DATA	GPIO21, physical pin 40
SCK	Usually ground or unused on modules with 3-wire I2S PLL; verify module instructions

Some modules expose XSMT, FMT, FLT and DEMP pads. Do not guess their state; follow that module's documentation. Confirm the board accepts 3.3 V I2S logic.

For a separate stereo jack:

Jack contact	Signal
Tip	Left output
Ring	Right output
Sleeve	Audio ground

10. Buttons and indicator wiring

Suggested assignments avoid SPI and I2S pins:

Control	GPIO	Physical pin	Wiring
Trigger button	GPIO5	29	Button to ground; enable internal pull-up
Continuous mode switch	GPIO6	31	Switch to ground; enable internal pull-up
Waveform/mode switch	GPIO16	36	Switch to ground; enable internal pull-up
Pulse LED	GPIO13	33	GPIO through 330 ohm resistor to LED, then ground

Debounce switches in software for 10-30 ms.

11. Control mapping

ADC channel	Panel label	Suggested range and function
CH0	Pitch	40-3000 Hz main oscillator, mapped exponentially
CH1	Speed	0.05-20 Hz modulation/LFO rate, mapped exponentially
CH2	Mod	0-100% pitch-sweep depth
CH3	Tone	Waveform blend or low-pass cutoff
CH4	Echo	20-800 ms delay time
CH5	Feedback	0-92% delay feedback/reverb amount
CH6	Volume	Muted to safe line-level maximum
CH7	Expansion	Envelope decay, noise mix or preset selector

Smooth every ADC channel. A simple one-pole smoother is sufficient:

```
smoothed = smoothed + coefficient * (new_reading - smoothed)
```

A coefficient around 0.05-0.2 at a 100-200 Hz control rate provides responsive controls without audible zipper noise.

12. Sound engine

Run audio at 48 kHz stereo with a 128- or 256-frame buffer. Begin at 256 for reliability; reduce only after testing.

Oscillator

Maintain a normalized phase accumulator from 0 to 1:

```
phase = wrap(phase + frequency / sample_rate)
```

Generate selectable sine, triangle, saw and square waves. Add a small amount of band-limiting or filtering to reduce harsh aliasing at high pitch.

Siren modulation

Use a second low-frequency oscillator:

```
instant_frequency = base_frequency * 2^(depth_octaves * lfo_value)
```

Exponential modulation sounds more musical than adding a fixed number of hertz.

Envelope

The trigger button starts a fast attack and adjustable decay/release envelope. Continuous mode holds the envelope open. Apply the envelope before delay so echoes continue after the dry siren stops.

Tone section

Start with a one-pole low-pass filter. Later, let the tone knob blend waveform shape and filter cutoff for a wider range of sounds.

Echo and reverb

Implement a circular delay buffer:

```
delayed = delay_buffer[read_position]
delay_buffer[write_position] = input + delayed * feedback
output = dry * input + wet * delayed
```

Clamp feedback to approximately 0.92 in normal mode. Smooth delay-time changes or crossfade between read positions to avoid clicks. A small network of short all-pass delays can add reverb after the main echo, but the first playable version only needs the main feedback delay.

Output safety

- Add a soft clipper or limiter after the effects.
- Start with master gain at -18 dB.
- Remove DC offset with a high-pass filter around 10-20 Hz.
- Test on an inexpensive powered speaker before a valuable mixer or headphones.

13. Software structure

Recommended processes/threads:

```
audio callback, real-time priority
- oscillator and modulation
- envelope and filter
- delay/reverb
- limiter and stereo output

control loop, 100-200 Hz
- MCP3008 reads
- smoothing and parameter mapping
- buttons, debounce and LEDs

service/management
- startup, logging and graceful shutdown
```

Do not print logs, allocate large objects or read files inside the audio callback.

Implementation choices

1. Python prototype: Python, NumPy, spidev, GPIO library and an ALSA-compatible audio library. Fastest to develop, adequate for the simple first engine.
2. Pure Data: visual patching, mature audio timing and quick musical experimentation.
3. C++/ALSA: best control over latency and CPU use for the final appliance.

Recommended path: prove the sound in Python or Pure Data, then keep it if underruns remain at zero during a long test. Rewrite only if reliability requires it.

14. Operating-system setup

1. Install Raspberry Pi OS Lite 64-bit with Raspberry Pi Imager.
2. Preconfigure Wi-Fi, hostname and SSH in the imager.
3. Boot, update packages, and change the default password if not already configured.
4. Enable SPI for the MCP3008.
5. Configure the selected USB or I2S audio device.
6. Disable unnecessary desktop services by using the Lite image.
7. Install the application and dependencies in a dedicated directory.
8. Create a `systemd` service that starts after the audio device is available.
9. Configure automatic restart after failure.
10. Keep the root filesystem cleanly shut down; add a shutdown button before treating the instrument like an appliance.

Example service behavior:

- Start synth after audio initialization.
- Retry if DAC is temporarily unavailable.
- Write only short diagnostic logs.
- Fade output to zero during shutdown.

15. Build phases

Phase 1 - software-only audio proof

1. Install the Pi and USB sound adapter.
2. Generate a fixed oscillator tone.
3. Verify clean stereo output to an active speaker at low volume.
4. Add LFO pitch modulation and a basic delay.

Exit criterion: continuous 30-minute playback with no underruns, crashes or loud startup pops.

Phase 2 - one control

1. Place MCP3008 and one 10 kohm pot on the breadboard.
2. Enable SPI and confirm raw readings near 0-1023.
3. Smooth and map the pot to pitch.
4. Check that the full rotation has no dead region or unstable endpoint.

Exit criterion: smooth pitch control without audible stepping.

Phase 3 - complete control panel

1. Add all seven pots.
2. Add trigger button, switches and LED.
3. Label every wire and control.
4. Confirm there are no GPIO conflicts.
5. Add calibration limits in software.

Exit criterion: all controls operate independently and recover correctly after reboot.

Phase 4 - final I2S audio

1. Add PCM5102A module.
2. Configure I2S overlay and select it as default ALSA output.
3. Repeat noise, latency and long-run tests.
4. Compare output level with the USB adapter.

Exit criterion: clean 3.5 mm line output, no audible digital noise during knob movement, zero underruns for two hours.

Phase 5 - enclosure

1. Freeze knob and switch positions before drilling.
2. Mount Pi, ADC/perfboard and DAC on insulated spacers.
3. Keep mains adapters outside the enclosure.
4. Separate audio-output wiring from power and SPI wiring.
5. Add strain relief and ventilation.
6. Provide access to microSD and power.

Exit criterion: no exposed conductors, secure controls, reliable startup and no cable-induced noise.

16. Test plan

Test	Method	Pass condition
Power	Measure 5 V and 3.3 V rails before connecting modules	Correct voltage and polarity
ADC endpoints	Log all channels while rotating each pot	Stable near 0 and 1023; no cross-channel movement
Switches	Trigger each input 50 times	No missed or double events
Audio idle noise	Listen/record with oscillator muted	No loud hum, ticking or digital chatter
Output level	Begin at -18 dB and raise slowly	Clean mixer/active-speaker input without clipping
Delay stability	Maximum normal feedback for five minutes	No uncontrolled numerical growth
CPU	Monitor load during worst-case settings	Comfortable headroom; no sustained 100% core
Underruns	Run for two hours and inspect logs	Zero ALSA underruns
Reboot	Perform ten cold boots	Synth starts and controls work every time
Shutdown	Use software shutdown control	No corrupted SD card or loud pop

17. Troubleshooting

Symptom	Likely cause	Action
No knob movement	SPI disabled, wrong CS pin or missing common ground	Verify SPI device, GPIO numbering and MCP3008 pins 9/14
Readings jump	Long wires, floating channel or noisy 3.3 V rail	Shorten wires, add decoupling and smoothing
Constant maximum ADC	Pot wired to 5 V or VREF mismatch	Disconnect immediately; use 3.3 V only
No I2S audio	Wrong overlay, pin mapping or DAC jumper state	Test ALSA device directly and check module documentation
Distorted output	DAC feeding headphones, software clipping or wrong mixer gain	Use line input, lower master gain and enable limiter
Clicks during echo changes	Delay read pointer jumps	Smooth or crossfade delay-time changes
Periodic audio gaps	Buffer too small or background load	Increase to 256/512 frames and reduce services
Hum	Ground loop or audio cable near power wiring	Use one ground reference and reroute/shield audio cable

Symptom	Likely cause	Action
Startup pop	DAC unmuted before stream is stable	Hold mute/fade at zero during initialization

18. Risks and mitigations

Risk	Mitigation
Linux scheduling causes glitches	OS Lite, fixed audio buffer, real-time priority and no work in callback
SD corruption after power removal	Shutdown button, read-only filesystem later, or controlled power switch
Incorrect DAC module purchased	Require I2S PCM5102A module with documented Raspberry Pi compatibility
Line output mistaken for speaker output	Label jack LINE OUT; use active speaker/mixer
GPIO damage	3.3 V-only control rail and continuity checks before powering
Control noise	Decoupling, short wiring, ADC smoothing and software dead bands
Scope growth	Complete basic oscillator/LFO/delay instrument before presets or displays

19. Definition of done

The digital dub siren is complete when:

- Seven knobs control pitch, speed, modulation, tone, echo, feedback and volume.
- Trigger and continuous modes work without missed events.
- Audio exits through a labelled 3.5 mm line-output jack.
- Startup volume is safe and there are no loud pops.
- The system boots directly into the synth without keyboard or monitor.
- A two-hour stress test produces no audio underruns or crashes.
- Power and GPIO wiring are enclosed and strain-relieved.
- Source code, wiring map, setup steps and final component choices are committed to this repository.

20. Future extensions

- Preset storage and recall.
- MIDI input over USB or TRS.
- Tap tempo and tempo-synchronised echo.
- More waveforms, sample playback and noise bursts.
- OLED display for values and preset name.
- Web interface over Wi-Fi.
- Record loops to microSD.
- Stereo ping-pong delay.
- Custom control PCB after the breadboard design is stable.

21. Recommended next action

Buy or borrow only the Pi Zero 2 WH, microSD, 5 V supply and a simple USB sound adapter first. Implement the fixed oscillator, modulation and echo before purchasing the complete control panel. This validates the software approach and audio latency at the lowest cost. Add the MCP3008 and one pot next, then expand to seven controls and the final PCM5102A DAC.